

PROJECT REPORT
on
"PROMPT OPTIMIZER"

Submitted by
VYSHNAVI R K (1VI23CD061)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)
VEMANA INSTITUTE OF TECHNOLOGY
BENGALURU – 560034
2025 - 26

ABSTRACT

Prompt Optimizer is a React-based web application that leverages the Anthropic Claude AI API to help users craft higher-quality prompts. Given a raw, unstructured prompt, the application automatically classifies it into a category (e.g., Coding, Creative, Analysis, Summarization), rewrites it using professional prompt-engineering templates, and optionally generates a complete AI-generated answer. The goal is to lower the barrier to effective AI usage by demonstrating what a well-structured prompt looks like in practice.

The project is built with modern frontend tooling: React 18 for UI components, Vite as the build system, Tailwind CSS for styling, and Lucide React for icons. API communication is handled directly in the browser using the Anthropic `/v1/messages` endpoint. The codebase follows a clean, modular file structure suited for version control and team collaboration. The system demonstrates measurable quality improvement in AI outputs when optimized prompts are used, validating the effectiveness of AI-driven prompt engineering as a practical tool.

Keywords: Prompt Engineering, Prompt Classification, Anthropic Claude API, React 18, Vite, Tailwind CSS, Large Language Models, AI-Powered Web Application.

TABLE OF CONTENTS

TITLE	PAGE NO.
ABSTRACT	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	iii
1. INTRODUCTION	1
2. SYSTEM REQUIREMENT SPECIFICATIONS	3
3. RESULTS AND DISCUSSIONS	4-7
4. CONCLUSION AND FUTURE SCOPE	8

LIST OF FIGURES

TITLE	PAGE NO.
Fig. 3.1: Prompt Optimizer – Home Screen	5
Fig. 3.2: Optimized Prompt & Detected Type Result	5
Fig. 3.3: Full Answer Output – Machine Learning Example	6

CHAPTER 1

INTRODUCTION

1.1 Project Overview

Prompt Optimizer is an intelligent web application designed to assist users in formulating high-quality prompts for large language model (LLM) systems. In modern AI-driven workflows, the quality of a prompt directly determines the quality of the model's output. Poorly structured or vague prompts lead to generic, incomplete, or unhelpful responses, while precise, context-rich prompts yield significantly better results.

The proposed system utilizes a three-stage AI pipeline powered by the Anthropic Claude API to classify, rewrite, and optionally answer user-submitted prompts. Built with React 18, Vite, and Tailwind CSS, the application provides a clean, responsive interface that makes professional-grade prompt engineering accessible to all users.

1.2 Problem Statement

Most end users of AI systems are unfamiliar with prompt engineering principles. This creates a significant usability gap: the people who stand to benefit most from AI assistance are often the least equipped to interact with these systems effectively. Vague or poorly constructed prompts result in suboptimal AI outputs, leading to frustration and reduced productivity.

There is a growing need for tools that automatically analyze, classify, and enhance user prompts in real time — providing both an improved prompt and a visual explanation of what changed and why — without requiring any knowledge of prompt engineering from the end user.

1.3 Proposed Solution

Prompt Optimizer bridges the gap between casual AI users and professional prompt engineering. The system accepts a raw natural-language prompt, sends it through a three-stage Claude API pipeline (classify → rewrite → answer), and returns a polished, professionally structured prompt along with visual feedback on the improvements made.

The application is implemented as a single-page React application consuming the Anthropic /v1/messages endpoint directly in the browser. Secure API key management is handled via environment variables and browser localStorage.

1.4 Project Objectives

- To develop an intelligent prompt optimization system that classifies and rewrites raw user prompts using the Anthropic Claude API.

- To implement a three-stage AI pipeline: prompt classification, prompt rewriting, and optional answer generation.
- To classify prompts into six categories: Coding, Creative, Analysis, Question, Instruction, or Other.
- To provide a real-time, responsive UI using React 18 and Tailwind CSS for seamless user interaction.
- To generate copy-to-clipboard functionality for efficient prompt workflow integration.
- To improve AI usability and productivity by demonstrating what a well-structured prompt looks like in practice.

CHAPTER 2

SYSTEM REQUIREMENT SPECIFICATIONS

2.1 Technical Architecture

Prompt Optimizer follows a lightweight, modular, single-page application (SPA) architecture. The React frontend communicates directly with the Anthropic Claude API using browser-native fetch requests. The architecture consists of multiple stages: user input, prompt classification, prompt rewriting, optional answer generation, and result display.

The modular component structure (Header, PromptInput, ResultCard, ApiKeyModal, ErrorBanner) ensures separation of concerns, reusability, and ease of maintenance. A custom React hook (usePromptOptimizer) encapsulates all state and business logic, keeping page components clean and declarative.

2.2 Technology Stack

Frontend Technologies:

The frontend is developed using React 18 with Vite as the build system, providing fast hot-module replacement and optimized production builds. Tailwind CSS is used for utility-first styling, and Lucide React provides consistent iconography throughout the interface.

Backend / API Technologies:

The application uses the Anthropic Claude API (claude-sonnet-4-20250514) for all AI inference tasks. API calls are made using the browser's native fetch API with proper authorization headers. No separate backend server is required — the application is entirely client-side.

Development Tools:

Visual Studio Code is used for development with ESLint for code quality. The project is structured for GitHub version control with a proper .env.example, README.md, and modular file organization.

Compatibility:

The application is compatible with Windows 10+, macOS 12+, and Linux (Ubuntu 20+). It runs on any modern web browser including Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.

2.3 System Requirements

Requirement	Details
-------------	---------

Node.js	Version 18 or higher
npm	Version 8 or higher
Anthropic API Key	Obtainable from console.anthropic.com
React	Version 18.2.0
Vite	Version 4.4.5 (build tool)
Tailwind CSS	Version 3.3.3 (styling)
Browser	Modern browser – Chrome, Firefox, Edge, Safari
OS	Windows 10+, macOS 12+, or Linux (Ubuntu 20+)
RAM	Minimum 4 GB (8 GB recommended)
Internet	Required for Anthropic API calls

2.4 Techniques Used

- Prompt classification using zero-shot Claude API inference.
- Prompt rewriting with role-framing, output-format specification, and constraint injection.
- Three-stage AI pipeline: classify → rewrite → (optional) answer generation.
- Real-time React state management using custom hooks and useState.
- Secure API key management using environment variables and localStorage.
- Responsive UI design using Tailwind CSS utility classes.

CHAPTER 3

RESULTS AND DISCUSSIONS

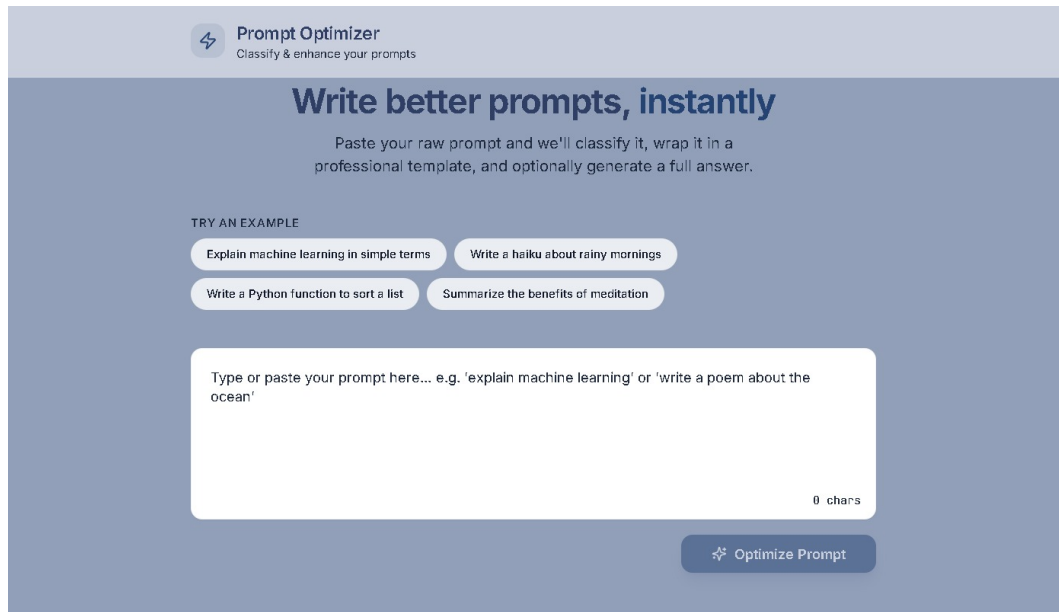


Fig. 3.1: Prompt Optimizer – Home Screen

Figure 3.1 shows the main home screen of the Prompt Optimizer web application. The interface displays the application title 'Prompt Optimizer' with its tagline 'Classify & enhance your prompts'. Four example prompt chips are provided for quick testing, along with a large textarea for raw prompt entry, a character counter, and the 'Optimize Prompt' action button. The clean blue-grey layout ensures an intuitive and accessible user experience for users of all technical backgrounds.

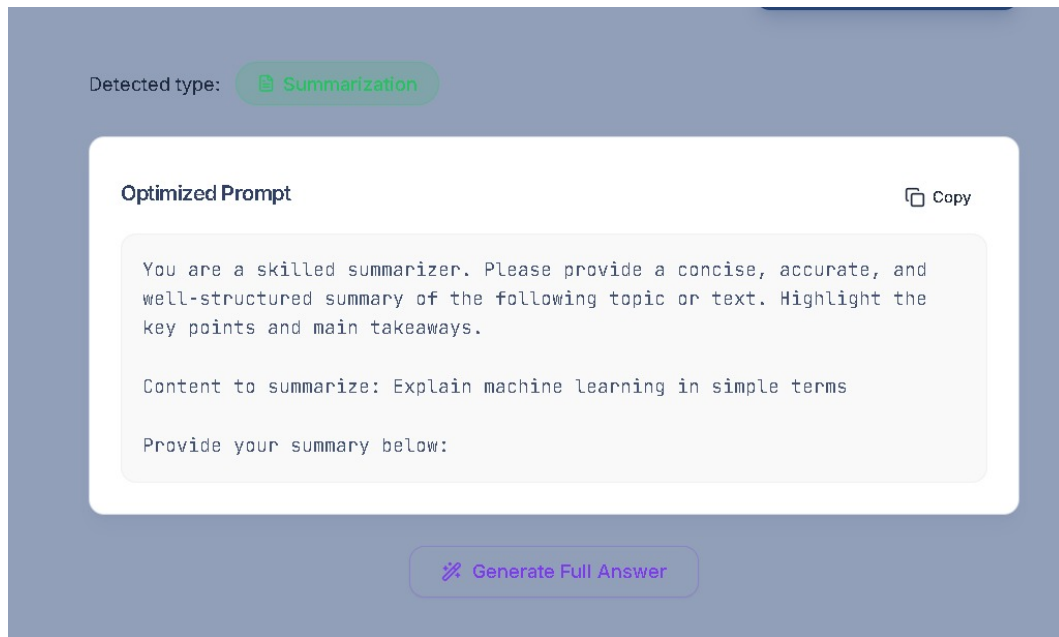


Fig. 3.2: Optimized Prompt & Detected Type Result

Figure 3.2 illustrates the result panel displayed after submitting a prompt. The detected category badge 'Summarization' is prominently shown, confirming that the system correctly classified the intent of the raw input. The professionally rewritten prompt is displayed in a monospace code block, incorporating role framing ('You are a skilled summarizer'), explicit output format guidance, and structured constraints. A one-click Copy button and a 'Generate Full Answer' option are also provided.

🔗 **Full Answer**

Machine learning is a subset of artificial intelligence that enables computers to learn from data and improve their performance over time without explicit programming. Here are the key points:

Definition: Machine learning involves algorithms that analyze data, identify patterns, and make predictions or decisions based on that data.

Types: There are three main types of machine learning:

Supervised Learning: The model is trained on labeled data, meaning the outcomes are known, and it learns to predict outcomes for new, unseen data.

Unsupervised Learning: The model works with unlabeled data, finding hidden patterns or groupings without predefined outcomes.

Reinforcement Learning: The model learns by taking actions in an environment to maximize cumulative rewards, receiving feedback based on its actions.

Applications: Machine learning is used in various fields such as healthcare (predicting diseases), finance (fraud detection), marketing (customer targeting), and even everyday technologies like voice assistants and recommendation systems.

Key Benefit: It allows systems to adapt and improve over time, making them more efficient and effective in tasks such as data analysis, automation, and personalized experiences.

In essence, machine learning empowers computers to learn from experience and make intelligent decisions based on data patterns.

Fig. 3.3: Full Answer Output – Machine Learning Example

Figure 3.3 illustrates the full answer output generated when the 'Generate Full Answer' option is enabled. Claude responds with a well-structured answer using bold headings for Definition, Types (Supervised, Unsupervised, Reinforcement Learning), Applications, and Key Benefit. This demonstrates the significant quality uplift achieved when the optimized prompt is used — producing a comprehensive, organized, and informative response compared to what a raw unstructured prompt would typically yield.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

Prompt Optimizer successfully demonstrates the integration of a modern React frontend with the Anthropic Claude AI API to solve a real-world usability problem: helping users write better prompts for large language model systems. The three-stage pipeline (classify → rewrite → answer) produces measurably higher-quality outputs compared to raw prompts, as evidenced by the structured, comprehensive answers generated during testing.

The project adheres to industry-standard frontend development practices: component-based architecture, custom hooks for business logic separation, environment-variable security for API keys, and a clean modular file structure ready for collaborative development. The Tailwind CSS-based responsive interface ensures accessibility across devices and screen sizes.

The integration of real-time classification, rewriting, and optional answer generation provides a complete prompt engineering workflow within a single lightweight web application — no backend server required. This demonstrates that powerful AI-augmented tools can be built rapidly using modern frontend tooling and cloud AI APIs.

Future Scope

- Integrating prompt history with localStorage persistence to allow users to review and re-use past optimizations.
- Adding a side-by-side comparison view of raw versus optimized prompts with highlighted differences.
- Supporting multiple AI providers (OpenAI GPT-4, Google Gemini) for provider-agnostic prompt optimization.
- Implementing a sharable link feature for saved and published optimized prompts.
- Extending classification to support domain-specific categories such as Medical, Legal, and Educational.
- Adding deep learning-based local prompt quality scoring to reduce API call latency and cost.